

Trainer Teaching Guide: Spring Boot Session 2

- REST Web Services

A detailed, textbook-style script for [springboot-session2-theory.html](#). Simple English, examples, analogies and natural fillers - read it aloud and it sounds like teaching, not reading.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [springboot-session2-theory.html](#) (20 slides)

HOW TO USE THIS GUIDE:

- ON SCREEN tells you what the slide shows.
- SAY is the full teaching script - read it word-for-word; fillers and questions are built in.
Pause where you see '...'.
- EMPHASISE / DELIVERY TIP are notes for you - do NOT read these aloud.

Slide 1 - Title: REST Web Services

ON SCREEN

The blue cover slide: 'Spring Boot', with the line about building a complete Student CRUD API.

SAY (read this aloud)

Welcome back, everyone. Last session, we got an app up and running, and it said hello to us. That was a big first step - but right now our app can only really talk to a human looking at a browser. Today, we level up. Today we make our app talk to other programs.

That is what a web service is - a program that other programs can call to get data. Think about your food delivery app. When it shows you restaurants, it is not magic - it is quietly calling a web service in the background and getting back a list of data. Today, you learn to build exactly that kind of service.

We are going to build a complete Student system - the ability to create a student, read students, update one, and delete one. And here is the honest truth about today: the theory part is short, because the real learning happens in the lab, where you build the whole thing yourself, one piece at a time. So let me set up the ideas quickly, and then we go and build.

DELIVERY TIP

Keep the opening short and punchy. The energy of Session 2 should be 'let's build' - use a relatable web-service example like a food app or weather app to make it concrete.

Slide 2 - Agenda

ON SCREEN

Six agenda items: Session 1 recap, web services and JSON, REST vs SOAP and HTTP, URI design and annotations, project setup, and the hands-on lab.

SAY (read this aloud)

Here is our path for today. First, a quick three-question warm-up from Session 1, just to wake up the brain. Then we will understand what a web service really is, and the language it speaks - something called JSON. Then we talk about REST, the popular style we will follow, and the HTTP methods and status codes that go with it. Then how to design clean web addresses, and the annotations you will use. A quick word on project setup. And then the main event - building the Student API live, together, in the lab.

So just a little theory to give you the vocabulary, and then your hands are on the keyboard for the rest. Let's get that warm-up going.

Slide 3 - Session 1 recap

ON SCREEN

Three click-to-reveal flash cards on @SpringBootApplication, @RestController, and the default port.

SAY (read this aloud)

Quick warm-up - three questions. I will read each one, you answer in your head, then I will reveal it on the card. Ready?

Question one: what does @SpringBootApplication do? Think... Okay, reveal. It is one annotation that bundles three together, and it marks the main class that starts your whole application. Good.

Question two: what is @RestController? Think about it... Reveal. It marks a class that handles web requests, and importantly, whatever it returns goes straight back to the caller as data - not as a web page. That word 'data' is the theme of today.

And question three, an easy one: what port does Spring Boot use by default? Reveal - port 8080. You saw that line in your console last time, 'Tomcat started on port 8080'.

If all three felt comfortable, you are ready. If one was fuzzy, no problem - it will come back as we use it today.

DELIVERY TIP

Make this lively and verbal - call on a couple of people for each one before revealing. It re-activates last session's vocabulary, which today builds directly on.

Part 1 - Web Services & Data

Slide 4 - What is a web service?

ON SCREEN

A definition, plus a two-column diagram contrasting a traditional web app returning HTML with a web service returning JSON.

SAY (read this aloud)

So let's nail down what a web service actually is. A web service is a way for two applications to talk to each other and exchange data over a network - and it does not matter what languages they are written in. A Java app can talk to a Python app, which can talk to a phone app. The key word, again, is data.

Look at the two columns on screen, because this contrast is the whole idea. On the left is a traditional web app. You ask it for a page, and it sends back HTML - a full web page, designed for a human's eyes, with colours and layout. On the right is a web service. You ask it for something, and it sends back pure data - usually JSON - designed for another program to read and use, not for a human to look at.

Let me give you an analogy. A traditional web app is like a fully plated, decorated dish served at your table - ready to eat, made to look nice. A web service is like raw ingredients delivered to a

chef - no decoration, just the data, and the chef, which is the other program, does whatever it wants with it. Same kitchen underneath; completely different customer. Today, we are building the ingredient supplier.

Slide 5 - JSON syntax

ON SCREEN

A single JSON object for a student, then a JSON array of students, with the parts colour-coded.

SAY (read this aloud)

Now let's meet the language web services speak - JSON. It stands for JavaScript Object Notation, but do not worry about the name. What matters is that it is a simple, lightweight text format for data, and every programming language can read it. You will be staring at JSON in your testing tool in about twenty minutes, so let me make it click now.

Look at the single student on screen. See the curly braces? They wrap one object - one record, one student. Inside, we have pairs - a key on the left, always in quotes, then a colon, then a value. Like 'firstName' colon 'Ravi'. Commas separate the pairs. And the values can be text in quotes, plain numbers, or true-slash-false for yes-no.

Now the second example uses square brackets - those make a list, or array. So a response with several students is just a list of those objects. Easy.

And here is the part that will make you smile - you will not write any of this by hand. Spring Boot uses a library called Jackson that automatically turns your Java Student object into JSON on the way out, and turns incoming JSON back into a Java object on the way in. You hand it a Java object, the caller receives JSON. That automatic conversion has a name - serialization - and Spring does it for you.

Slide 6 - Key web service terminology

ON SCREEN

Four cards with analogies: service provider, service consumer, synchronous, asynchronous. A key box on loose coupling.

SAY (read this aloud)

A few vocabulary words you will hear constantly - and each comes with an everyday analogy, so they stick. Think of a restaurant.

First, the service provider - the app that offers the service and answers requests. That is the kitchen that cooks the food. Today, your Spring Boot API is the provider. Second, the service consumer - the app that calls the service and uses its data. That is the customer who orders. Your testing tool, a browser, or a phone app are consumers. Third, synchronous - the consumer asks and then waits for the answer before doing anything else. That is standing at the counter until your order is handed to you. And fourth, asynchronous - the consumer asks but does not wait; it carries on and gets notified later. That is taking a buzzer and going to sit down until it buzzes.

EMPHASISE

Land the 'loose coupling' point from the key box, because it is the real superpower. The two sides only agree on the data format and the addresses. So a Python app, a React website, and an Android phone can all call your one Java API. Change one side's internals and the other does not break. That independence is why web services scale across whole companies.

Part 2 - REST, HTTP & Design

Slide 7 - REST vs SOAP

ON SCREEN

A comparison table of SOAP versus REST across what it is, data format, weight, complexity, contract, transport.

SAY (read this aloud)

There are two main styles for building web services, and you should know both names. SOAP is the older, heavier one, based on XML. REST is the lighter, more flexible style that dominates today - and the one we will use. Let me be fair though - SOAP is not the villain; it is just from an earlier era.

Read across the table with me. SOAP is a strict protocol with rigid rules; REST is a flexible style with guidelines. SOAP only speaks XML, which is bulky; REST usually uses JSON, which is light. SOAP is heavier and slower; REST is lightweight and fast. SOAP needs a formal contract file; REST does not.

Here is where each lives today, so you have context. REST is everywhere now - web, mobile, microservices. SOAP still survives in older banking, payments, and big government systems, where its strict rules and built-in security were valued. So you will meet REST constantly, and you will bump into SOAP occasionally in older codebases. Today, and for most of your career, REST is the answer.

Slide 8 - The six REST principles

ON SCREEN

A list of the six REST constraints: stateless, client-server, uniform interface, cacheable, layered system, code on demand.

SAY (read this aloud)

REST is defined by a set of rules, or constraints. Now, I am not going to make you memorise all six today - relax. But knowing they exist helps you understand why REST apps are built the way they are. Let me touch each one lightly.

Stateless - every request carries everything the server needs; the server remembers nothing between requests. Client-server - the front end and the data side are separate and can change independently. Uniform interface - a consistent, predictable way to reach things. Cacheable - responses can say whether they are safe to store and reuse. Layered system - there can be

helpers in the middle, like load balancers, without either side knowing. And the last one, code on demand, is optional and rarely used.

EMPHASISE

If they remember only ONE, make it stateless. Each request stands completely on its own; the server does not remember you between calls. Use an analogy: it is like a vending machine - you put in everything needed each time, and it does not care who you are or what you bought before. That one principle is why REST scales to millions of users.

Slide 9 - HTTP methods map to CRUD

ON SCREEN

A table mapping GET, POST, PUT, DELETE to Read, Create, Update, Delete with Student API examples.

SAY (read this aloud)

Now the practical heart of REST. REST uses the standard HTTP methods as verbs - little action words - and beautifully, the four main ones line up exactly with the four things you do with data, which we call CRUD: Create, Read, Update, Delete.

Let me map them. GET means read existing data - GET slash students lists all students. POST means create new data - POST slash students adds a student. PUT means update existing data - PUT slash students slash one updates student one. And DELETE means remove - DELETE slash students slash one.

Here is the mental model I want you to carry. Picture a student record. You GET it to look at it. You POST a new one when a student joins. You PUT to fix their email. And you DELETE when they leave. Notice the noun stays the same the whole time - slash students - and only the verb changes for each action. That is the entire mental model of a REST API. Same noun, different verbs.

Slide 10 - HTTP status codes

ON SCREEN

Colour-coded badges: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error.

SAY (read this aloud)

Every response from a web service comes with a little number called a status code, which tells the caller what happened. There are dozens, but you only need five today, and they come in friendly groups. Two-hundreds mean success. Four-hundreds mean the caller made a mistake. Five-hundreds mean the server broke.

Let me give you the five. 200 OK - it worked; Spring sends this automatically for a successful read. 201 Created - used after you successfully create something with POST; you will set this one yourself. 400 Bad Request - the caller sent something malformed, like broken JSON. 404 Not Found - the thing you asked for does not exist, like a student ID that is not there. And 500 Server Error - your own code crashed with an unhandled error; we will learn to handle those gracefully in Session 3.

Quick way to remember the groups - think of it like a report card: 2xx is 'all good', 4xx is 'you

messed up', 5xx is 'I messed up'.

Slide 11 - URI design principles

ON SCREEN

A good-versus-bad comparison: noun-based URLs versus verbs-in-the-URL, and a key box on the pattern.

SAY (read this aloud)

Let's talk about designing good web addresses - what we call URIs. There is one golden rule in REST, and if you remember it, your APIs will look professional: URIs are nouns, and HTTP methods are verbs. In plain words - do not put the action in the address. Let the method say what you are doing; the address just names the thing.

Look at the good examples on the left. slash students for the whole collection. slash students slash one for a specific student. And a query like question-mark name equals John for filtering. Clean, simple nouns.

Now the bad examples on the right - slash getStudent, slash deleteStudentById, slash student slash fetch. See the problem? The verb is jammed into the address, which is clumsy and redundant, because the HTTP method already says GET or DELETE. It is like naming a file 'open-this-file dot text' - the action does not belong in the name.

So the pattern to internalise: plural nouns for collections, a path piece to pick one item, and query parameters for optional filtering. The verb is never in the URL.

Slide 12 - Annotations you'll use today

ON SCREEN

A grid previewing ten annotations, from @RestController to @ResponseStatus.

SAY (read this aloud)

Here is your whole toolbox for the lab, laid out in one place. Don't worry about the details right now - we will unpack the important ones over the next few slides, and you will actually use each one in code. I just want you to see what is coming, so none of it feels strange later.

At-RestController and at-RequestMapping set up the controller and its base address. Then four mapping annotations - at-GetMapping, at-PostMapping, at-PutMapping, at-DeleteMapping - one for each verb. Then three for reading input: at-PathVariable pulls a value out of the address, at-RequestParam reads a query value, and at-RequestBody turns incoming JSON into a Java object. And finally at-ResponseStatus, which sets that status code number.

So that is ten annotations - and honestly, that is the entire surface area of building a REST controller. Ten little labels. You already know the first one from Session 1. Not so scary when you see it is a small, fixed set, right?

Slide 13 - @RestController unpacked

ON SCREEN

A note that @RestController equals @Controller plus @ResponseBody, with two code panels comparing them.

SAY (read this aloud)

Let me take the mystery out of at-RestController, because you have been using it without knowing what is inside it. It is actually a convenience that bundles two things together: at-Controller plus at-ResponseBody.

The important half is at-ResponseBody. This is the part that says: 'take whatever this method returns and send it straight back as data - as JSON - to the caller.' Without it, Spring would think your return value was the name of a web page to go and find. With it, your return value IS the response.

Look at the two panels. On the left, with a plain at-Controller, you would have to write at-ResponseBody on every single method - tedious and easy to forget. On the right, at-RestController applies it to all methods automatically. So for building APIs, where every method returns data, at-RestController is exactly what we want - it saves us from repeating ourselves on every method.

Slide 14 - @RequestMapping at class level

ON SCREEN

A before/after showing repeated /students paths collapsing into a class-level prefix, and a key box on URL assembly.

SAY (read this aloud)

Here is a neat little tidiness trick. When you put at-RequestMapping slash students at the top of the class - on the class itself - it sets a common prefix for every method inside. So each method's address is then just the part after slash students. You write slash students once, not on every method.

Look at the before-and-after. On the left, without it, every method repeats slash students - slash students, slash students slash all, slash students slash id - over and over. Repetitive, and if you ever rename it, you have to change it everywhere. On the right, with the class-level mapping, the methods just say the suffix - nothing, or slash all, or slash id. Much cleaner.

How does Spring build the full address? It simply glues the class prefix and the method part together. Class slash students, plus method slash id, equals slash students slash id. And the lovely bonus - change that prefix in one place and every endpoint moves with it automatically. It is like writing the street name once at the top of a list of house numbers, instead of on every line.

Part 3 - Tools & The Bigger Picture

Slide 15 - Testing tools: Postman & Swagger

ON SCREEN

Two cards - Postman and Swagger UI - plus a mock Swagger listing of endpoints.

SAY (read this aloud)

Now, a practical question - how do you actually test these endpoints? For a GET, you can just type the address in a browser. But you cannot easily test a POST or a PUT from a browser address bar, because browsers only do GET. So we use proper API testing tools, and you will meet two today.

The first is Postman. It is a dedicated app for calling APIs, and it gives you full control - you pick the method, set the headers, and paste in a JSON body. This is the industry-standard tool that real teams use every day. It is your best friend for testing POST, PUT, and DELETE.

The second is Swagger UI. This one is lovely because it builds itself automatically from your annotations - zero setup. You add one dependency, and Spring generates a live, interactive web page that lists every endpoint with a 'Try it out' button. Look at the mock on screen - that list of endpoints is generated from your controller; you write no documentation. Great for quick checks and for sharing your API with teammates.

Slide 16 - ResponseEntity

ON SCREEN

Two code panels: a raw return that is always 200, versus ResponseEntity that lets you choose the status.

SAY (read this aloud)

Let's talk about taking full control of your response. By default, when you return a Student object, Spring sends it back with status 200 OK. That is fine most of the time. But sometimes you need to control the whole response - the status code, the headers, everything. That is what ResponseEntity gives you - it is a wrapper around the entire response.

Look at the panels. On the left, a plain return - the status is always 200, even when the student was not found, which is not quite honest. On the right, using ResponseEntity, you can choose: return ResponseEntity.ok() with the student for a 200 when it exists, or ResponseEntity.notFound() for a clean 404 when it does not.

So when do you reach for it? Whenever the outcome can vary - a 201 Created after a POST, a 404 when something is missing, or when you need to attach special headers. Think of a plain return as always mailing a standard envelope, and ResponseEntity as choosing the envelope, the stamp, and the priority yourself. It is the professional way to build precise responses.

Slide 17 - RestTemplate

ON SCREEN

A RestTemplate code snippet calling another service, with bullets and a 'trainer demo only' warning.

SAY (read this aloud)

So far, your app has been the provider - it answers requests. But here is a twist: sometimes your app needs to be the consumer too. Sometimes your Spring app needs to call another web service from inside its own code. For that, Spring has a classic tool called RestTemplate.

Look at the snippet. The method `getForObject` sends a GET request to another service and automatically converts the JSON it gets back straight into a Java object. So your app can fetch data from somewhere else and use it. It is synchronous, meaning your code waits for the answer before moving on. And there are matching methods for the other verbs too.

DELIVERY TIP

This is a trainer demo only - tell them clearly not to code it today. Frame it as a teaser: 'this becomes really important in Session 4, when we have many small services that all call each other - that is microservices. For now, just understand what it is for.'

Slide 18 - Coming up in Session 3

ON SCREEN

Four cards previewing Session 3: exception handling, JPA plus H2, `@Entity`, `@Repository`.

SAY (read this aloud)

A quick look ahead, because today's app has two weaknesses we will fix next time. Right now our data lives in a list in memory - so it vanishes the moment the app stops. And if you ask for a student that does not exist, you just get an unhelpful empty result. Session 3 fixes both.

We will add proper exception handling, so a missing student returns a clean, friendly 404 message instead of an ugly crash. We will swap that in-memory list for a real database called H2, using Spring Data JPA, so your data survives restarts. We will map our Student class straight to a database table with one annotation, `@Entity`. And we will get full CRUD with almost no code, using a `@Repository`.

And the best part - everything you build today carries forward. Session 3 upgrades it from a demo into a real, persistent application.

Slide 19 - Annotations recap

ON SCREEN

The same ten annotations from the preview, now each with a one-line definition.

SAY (read this aloud)

Here is your closing cheat-sheet - the same ten annotations from earlier, now each with a short definition. Keep this slide; it is your revision anchor, because this really is the entire vocabulary of

building a REST controller.

Let me run through them quickly. `@RestController` - marks the controller, returns data as JSON. `@RequestMapping` - sets the base address. `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` - one for each of the four verbs, read, create, update, delete. `@PathVariable` - grabs a value from the address. `@RequestParam` - grabs a query value after the question mark. `@RequestBody` - turns incoming JSON into a Java object. And `@ResponseStatus` - sets the status code.

Ten labels. If you know what these ten do, you can build any REST API. That is genuinely the whole toolkit.

Slide 20 - Closing

ON SCREEN

The closing blue slide: 'Let's build it.'

SAY (read this aloud)

And that is the theory for today. You can now explain what a web service is, how REST uses nouns and verbs, what those status code numbers mean, and which annotations build a controller. That is a real, solid foundation.

Now let's open the lab and build the Student CRUD API together, one endpoint at a time - exactly the way real teams do it: write a method, run it, test it, move on. Don't just watch me - code along, because this stuff only truly clicks when your own fingers type it. Let's build it.

DELIVERY TIP

Transition straight into the lab while momentum is high. Encourage everyone to actually code along rather than watch - Session 2's value is entirely in the doing.